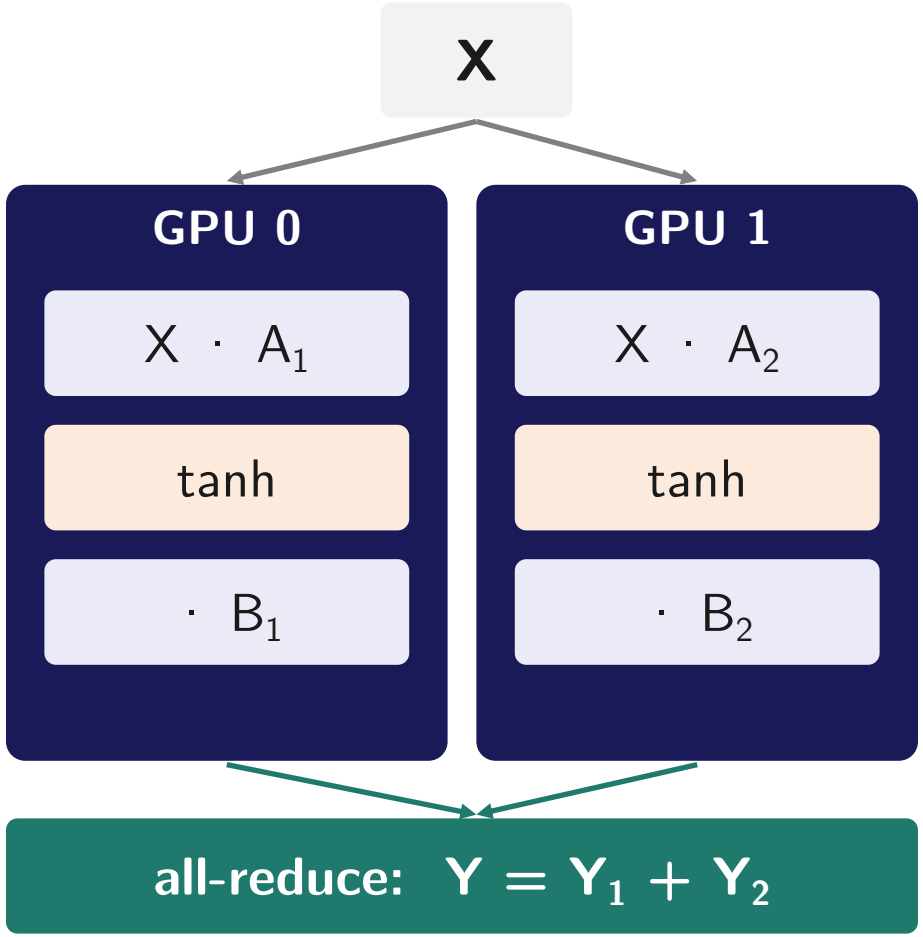
Tensor and Model Parallelism

Splitting one layer across GPUs: block matrices, two layouts, and one all-reduce

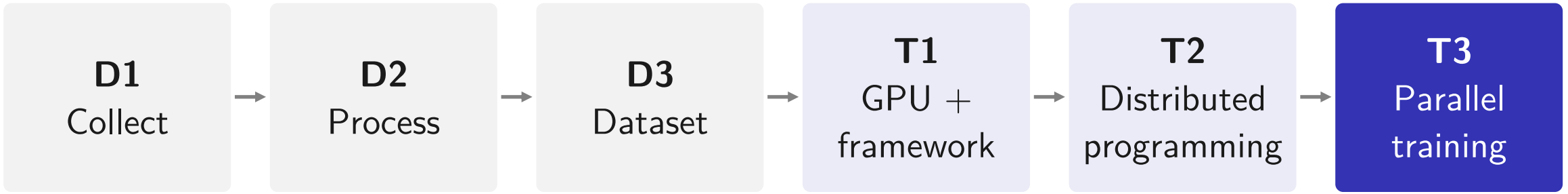
Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 2



Where we are in the pipeline



In our pipeline

Today: split **one layer** across GPUs. Lab 5 (tensor and pipeline partitions of a toy model) is released after this lecture.

Lecture 1: data parallelism, ZeRO, FSDP → today: tensor parallelism → Week 13: pipeline and 3D

Learning goals for today

- **Derive** column- and row-parallel matrix multiplies with block matrices.
- Say which **collective** each layout needs, in forward **and** in backward.
- Build the **Megatron-LM MLP**: one all-reduce forward, one backward (f and g).
- Estimate **communication per layer** from batch $\times$ sequence $\times$ hidden, and explain why tensor parallelism stays **inside a server**.
- Estimate **per-rank memory** for parameters and activations at TP degree t.

A puzzle to start

Two-layer MLP

$Y = \tanh(X\ A)\ B$

X: batch $\times$ features

Two GPUs

each holds half of A

and half of B

How few times must the GPUs communicate to compute Y?

Answer

Once. Split A by **columns** and B by **rows**: tanh stays local, and one all-reduce adds the two partial results.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

Recap: what data parallelism cannot fix

- **Memory**: ZeRO-3 still gathers each **full layer** and keeps its own activations.
- **Batch**: N GPUs need $\geq$ N examples per step; the global batch cannot grow forever.
- **Time per example**: each layer still runs on **one** GPU, start to finish.

Key idea

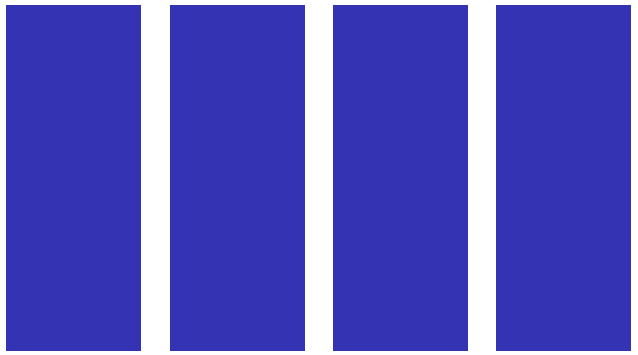
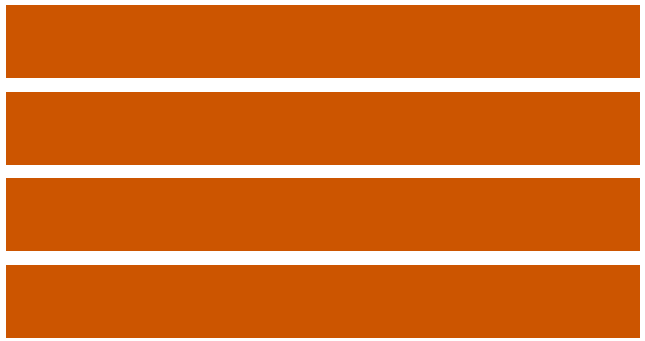
Tensor parallelism splits the **work inside each layer** — no larger batch needed.

Part 1

When the model is too big

Which dimension of the problem do we split?

Three ways to split training

Data	Tensor	Pipeline
		
split the batch	**split each layer's matrices**	**split the layer stack**
every GPU: whole model	every GPU: part of every layer	every GPU: some whole layers

Key idea

Today: **tensor** parallelism. Week 13: **pipeline**, and all three together (3D).

How big is one layer?

$$\text{MLP: } (h \times 4h) + (4h \times h) = 8h^2$$

$$\text{attention: } Q,\ K,\ V,\ \text{output} = 4h^2$$

$$\text{one layer} \approx 12h^2$$

h = 12,288	**per layer**	**96 layers**
parameters	1.81 B	174 B
Adam state	29 GB	2.8 TB

Watch out

One layer's optimizer state is over a **third** of an 80 GB GPU. The model must be split.

Source: Brown et al., "Language Models are Few-Shot Learners," 2020 (GPT-3: h = 12,288, 96 layers).

Activations grow with b · s · h

$$\text{one activation tensor: } b \times s \times h \text{ numbers}$$

$$b = 4,\ s = 2{,}048,\ h = 8{,}192\text{: } 2 \cdot b \cdot s \cdot h = 134 \text{ MB in bf16}$$

- Each layer keeps **several** such tensors for backward (the MLP's are 4h wide).
- Activations grow with the **batch**; parameters do not.
- **b · s · h** will also set the size of tensor parallelism's **messages**.

Source: Korthikanti et al., "Reducing Activation Recomputation in Large Transformer Models," 2022.

Check yourself

Two minutes with a neighbour.

- What does each GPU hold under data, tensor, and pipeline parallelism?
- A layer with $h = 4096$: how many parameters, and how many GB of Adam model state?
- You double the sequence length. What happens to parameter and activation memory?

Part 2

Partitioning a matrix multiply

Columns or rows: what does each GPU compute?

Block matrices: the one identity we need

$$X\text{: } m \times k, \quad A\text{: } k \times n, \quad Y = \mathrm{XA}\text{: } m \times n$$

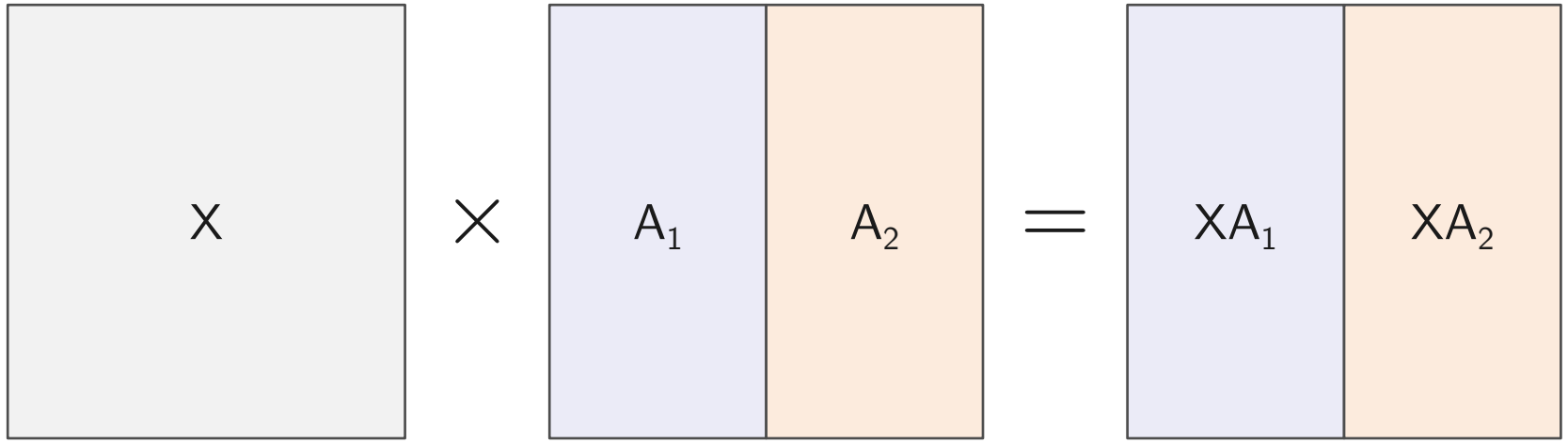
$$\text{columns: } \; A = [A_1 \;\; A_2] \quad \Rightarrow \quad \mathrm{XA} = [\mathrm{XA}_1 \quad \mathrm{XA}_2]$$

$$\text{rows: } \; A = [A_1 \; ; \; A_2], \;\; X = [X_1 \;\; X_2] \quad \Rightarrow \quad \mathrm{XA} = X_1A_1 + X_2A_2$$

Key idea

Column split $\rightarrow$ **pieces of Y**. Row split $\rightarrow$ **partial sums of Y**. Everything else follows.

Column-parallel: each rank owns a slice of Y



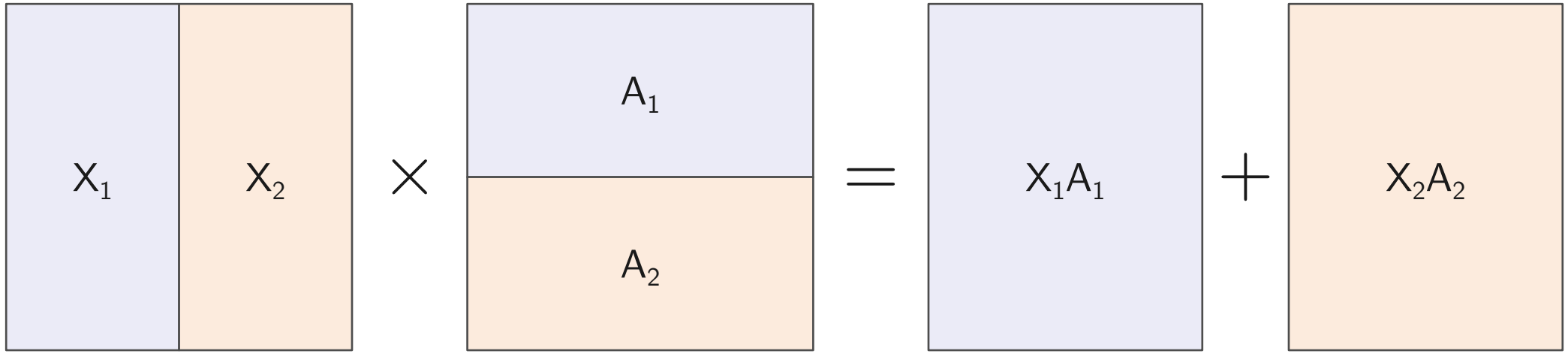
No communication to compute the pieces.

Each rank holds $\mathbf{1/t}$ of A and of Y.

Memory

Need the full Y on every rank? **All-gather** the slices. Often the next layer does not.

Row-parallel: each rank owns a partial sum



X by columns, A by rows

rank 0: full-size partial · rank 1: full-size partial

Memory

Every rank needs the **sum** → one **all-reduce** of a full-size Y. The input needs nothing if it already arrives split by columns.

The two layouts side by side

	Column-parallel	**Row-parallel**
Input X	replicated	split by columns
Output Y	split by columns	partial sums (full size)
Forward collective	none (all-gather if Y needed)	all-reduce
Weights, FLOPs	1/t per rank	1/t per rank

Key idea

Column-parallel **outputs** exactly what row-parallel **expects as input**. Chain them.

Which nonlinearities stay local?

$$\text{column split:}\quad \tanh([\mathrm{XA}_1 \quad \mathrm{XA}_2]) = [\tanh(\mathrm{XA}_1) \quad \tanh(\mathrm{XA}_2)]$$

$$\text{row split:}\quad \tanh(X_1A_1 + X_2A_2) \neq \tanh(X_1A_1) + \tanh(X_2A_2)$$

- Elementwise functions (tanh, GeLU, dropout) are **local** on column slices.
- On partial sums they need the **full sum first** $\rightarrow$ an all-reduce before them.

Watch out

Put the nonlinearity **after a column-parallel** layer, never after a row-parallel partial sum.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

What training adds: communication in backward

Layout	Forward	Backward (by autograd)
Column-parallel	no communication	one all-reduce (input gradient)
Row-parallel	one all-reduce (partial sums)	no communication

We take the framework's backward as given; only its **communication** matters here.

Memory

Column-parallel: free forward, **all-reduce in backward**. Row-parallel: **all-reduce in forward**, free backward.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

Check yourself

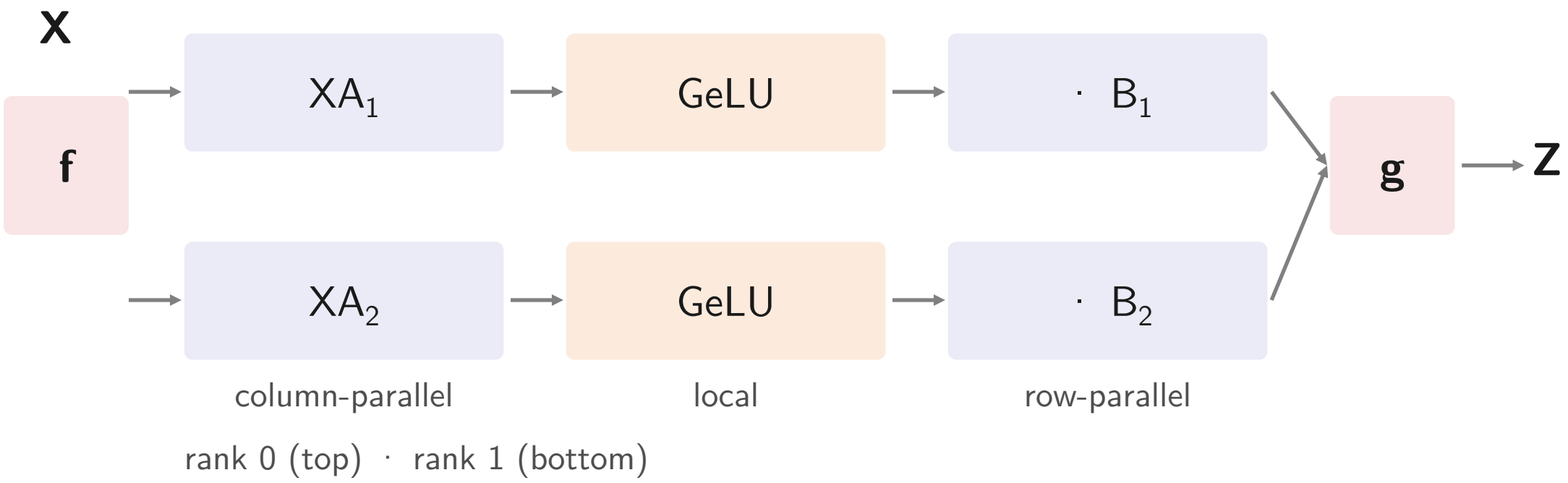
- Which split of A lets each rank compute its part of Y with **no** communication?
- Row-parallel needs X split by columns. Which layer produces X in that layout?
- Why can tanh not be applied to row-parallel partial sums?
- Which collective does a column-parallel layer need in **backward**, and on what tensor?

Part 3

The Megatron-LM MLP

One all-reduce forward, one backward — how?

Column, then row: the Megatron MLP



Key idea

Forward: **one all-reduce** (g) of the h-wide output. The 4h-wide hidden layer never moves.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

The conjugate operators f and g

Operator	Where	Forward	Backward
f	before the column-parallel layer	identity	all-reduce
g	after the row-parallel layer	all-reduce	identity

Each operator's backward is the other's forward; autograd inserts the backward collectives for us.

Memory

Per MLP block: **1 all-reduce forward + 1 backward**, each of size $b \times s \times h$.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

The Megatron MLP forward in PyTorch

```
def tp_mlp_forward(x, A_r, b_r, B_r, c):
    # x is replicated on every rank (f)
    h = torch.tanh(x @ A_r + b_r)  # column
    y = h @ B_r                    # row: partial
    dist.all_reduce(y)             # g: sum
    return y + c                   # bias once
```

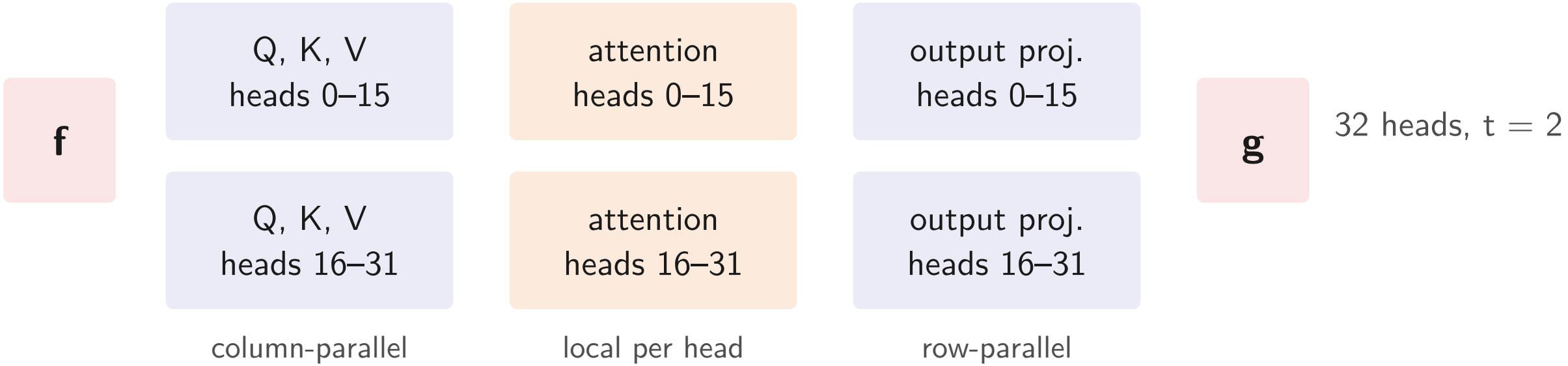
- Column layer: **local**.
- tanh: **local** on slices.
- Row layer: one **all-reduce**.

In practice

Training: `parallelize_module` wraps this layout; autograd adds the backward all-reduce. Real code passes the **tensor-parallel group**.

Source: Shoeybi et al. 2019; Megatron-LM source, megatron/core/tensor_parallel/mappings.py; PyTorch torch.distributed.tensor.parallel.

Attention: split the heads



Key idea

Heads are independent: each rank owns **1/t of the heads**. Again one all-reduce forward, one backward.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

One transformer layer under TP



	Forward	Backward	Size of each
All-reduces per layer	2 (g, g)	2 (f, f)	$b \times s \times h$

Memory

4 all-reduces per layer per step, each of $b \times s \times h$. LayerNorm and dropout run replicated.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. Korthikanti et al. 2022 (sequence parallelism).

Our MLP, split across ranks

Tensor	Full	Layout	$t = 2$	$t = 4$
Embedding table	322×16	replicated	322×16	322×16
W1, b1	128×64, 64	column	128×32, 32	128×16, 16
W2	64×322	row	32×322	16×322
b2	322	replicated	322	322
Parameters per rank	**34,338**		**19,906**	**12,690**

Watch out

Add b2 **once, after** the all-reduce — otherwise it is counted t times.

Check yourself

- What do f and g do in forward and in backward?
- How many all-reduces does one transformer layer need per training step?
- 64 attention heads and $t = 8$: how many heads per rank? Can $t = 3$ work?
- Why does our MLP not shrink to 34,338 / 4 parameters per rank at $t = 4$?

Part 4

Communication and memory

What does tensor parallelism cost, and where can it run?

Bytes per all-reduce

$$\text{message: } S = 2 \cdot b \cdot s \cdot h \text{ bytes } \quad (\text{bf16})$$

$$\text{per all-reduce, each rank sends } \quad 2(t-1)/t \cdot S$$

$$\text{per layer per step: } \quad 4 \times 2(t-1)/t \times S \;=\; 16\,(t-1)/t \cdot b{\cdot}s{\cdot}h$$

S = 134 MB $\rightarrow$ each rank sends 235 MB per all-reduce, 940 MB per layer

Memory

DP communicates **parameters** once per step; TP communicates **activations** 4 times per layer.

Our MLP: what tensor parallelism costs

Course MLP, t = 2, FP32	**Tensor**	**Bytes**
forward: g	logits 64×322	82,432
backward: f	∂X 64×128	32,768
compare — DP gradient all-reduce	34,338 parameters	137,352

- Bandwidth term: < 0.2 µs each at 450 GB/s.
- Latency term: $2(t-1)\alpha \approx 10$ µs **each**, twice per step ($\alpha \approx 5$ µs assumed).

Watch out

For our tiny model TP is **pure overhead** — useful only to check the algebra (Lab 5).

Communication versus computation

$$T(\text{comm}) \approx 16\ (t-1)/t \cdot b{\cdot}s{\cdot}h\ /\ \beta$$

$$T(\text{comp}) \approx 72\ b{\cdot}s{\cdot}h^2\ /\ (t\ \pi)$$

$$T(\text{comm})\ /\ T(\text{comp}) \approx (2/9)\ (t-1) \cdot \pi\ /\ (\beta\ h)$$

Key idea

b and s cancel. The ratio grows with **t** and π/β, and falls with **h**: tensor parallelism needs a **fast link**.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. (forward FLOPs per layer).

The numbers: NVLink versus the network

Link (one direction)	Per all-reduce	4 per layer	Compute per layer
NVLink, 450 GB/s	0.52 ms	2.1 ms	$\approx$ 10 ms
Network, 50 GB/s	4.7 ms	18.8 ms	$\approx$ 10 ms

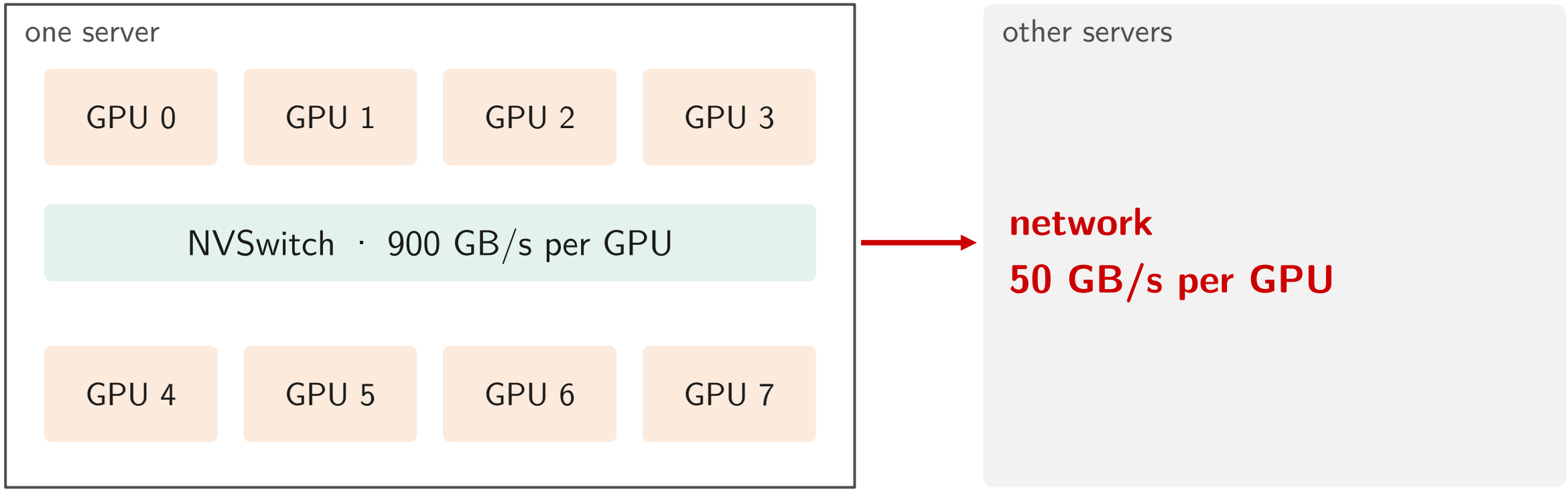
h = 8,192, s = 2,048, b = 4, t = 8, bf16; compute at 50% of 989 TFLOP/s

Watch out

NVLink: $\approx$ **20%** of compute. Network: $\approx$ **1.8×** the compute — and on the critical path.

Source: NVIDIA DGX H100 User Guide (8× ConnectX-7, 400 Gb/s per GPU); H100 datasheet (NVLink 900 GB/s bidirectional). 50% utilization assumed.

Why tensor parallelism stays in a server



Key idea

Rule: **t** $\leq$ **GPUs per server** (8). Across servers use **pipeline** and **data** parallelism.

Source: NVIDIA DGX H100 User Guide (8× ConnectX-7, 400 Gb/s per GPU); H100 datasheet (NVLink 900 GB/s bidirectional). Narayanan et al., SC 2021.

Per-rank memory: parameters

$$\text{model state per rank} \approx 16 \cdot 12h^2 \cdot L \;/\; t \text{ bytes}$$

h = 12,288, L = 96	**t = 1**	**t = 2**	**t = 8**
parameters per rank	174 B	87 B	21.7 B
model state per rank	2.78 TB	1.39 TB	348 GB

Watch out

t = 8 still leaves **348 GB** per GPU. TP alone is not enough: add **pipeline** and **ZeRO** (Week 13).

Per-rank memory: activations

$$\text{input } X\text{: } \ 2 \cdot b{\cdot}s{\cdot}h \quad \text{(replicated, does not shrink)}$$

$$\text{pre-activation + GeLU output: } \ 2 \times 2 \cdot 4 \cdot b{\cdot}s{\cdot}h \ / \ t$$

$$\text{MLP block: } \ 2 \cdot b{\cdot}s{\cdot}h \cdot (1 + 8/t) \text{ bytes}$$

h = 8,192, s = 2,048, b = 4: t = 1 → 1.21 GB, t = 8 → 0.27 GB per layer

Memory

The **replicated** input never shrinks with t; splitting it too is **sequence parallelism**.

Source: Korthikanti et al., "Reducing Activation Recomputation in Large Transformer Models," 2022.

Data, ZeRO-3, and tensor: what moves

	DP (DDP)	ZeRO-3 / FSDP	Tensor (TP)
Moves	gradients, 2Ψ	params + grads, 3Ψ	activations, 4 per layer
When	overlaps backward	prefetched per unit	critical path
Divides	nothing	all model state	weights, some activations
Needs batch $\geq$	N examples	N examples	1 example

Key idea

Large jobs use **all of them**: TP inside a server, data parallelism (and pipelines) across servers.

Check yourself

- $t = 4$, $b = 1$, $s = 2{,}048$, $h = 8{,}192$: how many MB does each rank send per all-reduce?
- What happens to the communication/computation ratio if h doubles?
- Why is $t = 16$ across two servers a bad idea?
- Which part of a rank's memory does **not** shrink as t grows?

IN-CLASS BUILD

A 2-way tensor-parallel MLP

Work in pairs. Part A: code it and check it. Part B: estimate per-rank memory.

Exercise A: column-then-row MLP forward

```
r, t = dist.get_rank(), dist.get_world_size()
torch.manual_seed(0)              # same weights
A, b = torch.randn(128, 64), torch.randn(64)
B, c = torch.randn(64, 322), torch.randn(322)
x = torch.randn(64, 128)
A_r = A.chunk(t, dim=___)[r]      # (1)
b_r = b.chunk(t)[r]
B_r = B.chunk(t, dim=___)[r]      # (2)
y = torch.tanh(x @ A_r + b_r) @ B_r
dist.all_reduce(y)                # (3) why?
y = y + c                         # (4) why here?
ref = torch.tanh(x @ A + b) @ B + c
torch.testing.assert_close(y, ref)
```

QUESTIONS

Fill in (1)–(2); explain (3)–(4).

What are the shapes of `A_r`, `B_r` and `y` on each rank?

STRETCH

Repeat with $t = 4$ on 4 ranks: what are the shapes, and does it still match?

Exercise B: per-rank memory

Quantity	Formula	t = 4	t = 8
parameters per layer	$12h^2$		
model state per GPU	$16 \cdot 12h^2 \cdot L \,/\, t$		
MLP activations per GPU	$2bsh\,(1 + 8/t) \cdot L$		
fits in 80 GB?			

GIVEN

$h = 8{,}192$, $L = 32$ layers, $s = 2{,}048$, $b = 1$, bf16 activations, 16 bytes per parameter. GB $= 10^9$ bytes.

Solutions

- (1) `dim=1`: `A_r` is 128×32.
- (2) `dim=0`: `B_r` is 32×322.
- (3) y is a **partial sum**: g adds them.
- (4) c is replicated: add it **once**.

Quantity	**t = 4**	**t = 8**
parameters per layer	805 M	805 M
model state per GPU	103.1 GB	51.5 GB
MLP activations	3.2 GB	2.1 GB
fits in 80 GB?	no	yes

Check

t = 8 fits, with room for attention activations; t = 4 needs ZeRO or pipeline help.

How to test it, and the library version

```
from torch.distributed.device_mesh import \
    init_device_mesh
from torch.distributed.tensor.parallel import (
    ColwiseParallel, RowwiseParallel,
    parallelize_module)

mesh = init_device_mesh("cuda", (2,))
mlp = parallelize_module(mlp, mesh, {
    "fc1": ColwiseParallel(),
    "fc2": RowwiseParallel()})
```

- **gloo on CPU** first: same answers.
- Compare with a **single-device** reference.
- Then $\mathbf{t = 4}$ (stretch).
- Library: same plan, **declared**.

In our pipeline

Lab 5: tensor and pipeline partitions of a toy model, bubbles, and a 3D layout.

Source: PyTorch documentation, torch.distributed.tensor.parallel (parallelize_module, ColwiseParallel, RowwiseParallel).

Key takeaways

- Column split of A $\rightarrow$ **slices of Y**, no communication; row split $\rightarrow$ **partial sums**, all-reduce.
- Elementwise nonlinearities are local on **column** slices: put them between column and row.
- Megatron MLP and attention: **f** (all-reduce backward) and **g** (all-reduce forward).
- 4 all-reduces of $\mathbf{b} \times \mathbf{s} \times \mathbf{h}$ per layer per step; cost/compute $\propto \mathbf{(t-1)}\ \pi\ /\ (\beta\ \mathbf{h})$.
- Keep $\mathbf{t} \leq$ **GPUs per server**: NVLink is $\approx 9\times$ the network bandwidth.
- Weights shrink as 1/t; replicated activations do not — and TP alone cannot fit the largest models.

Next time, and readings

When	What
After this lecture	Lab 5 released: tensor and pipeline partitions, bubbles, 3D layout
Week 13, Lecture 1	Pipeline and 3D parallelism
Week 13, Lecture 2	Diagnosing distributed systems

READ BEFORE WEEK 13

Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

Shoeybi et al., "Megatron-LM," 2019 — Section 3 (the MLP split).